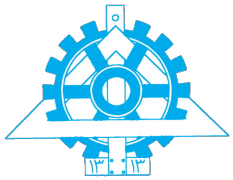
University of Tehran
College of Engineering
School of Electrical and
Computer Engineering



**Machine Learning**

# Final Project Phase 1

Full Name
Sepanta Ghonoodi

Student ID
810102483

February 16, 2026

## 0. Abstract

This report covers the theoretical and practical implementation of several core machine learning concepts, including clustering, dimensionality reduction, and feature selection. The first section explores unsupervised learning through a manual iteration of the K-Means algorithm and a comparison of Agglomerative Clustering linkage strategies, specifically focusing on how they handle computational complexity and outliers.

The second part of the assignment deals with dimensionality reduction. We analyze the different properties of Principal Component Analysis (PCA), Kernel PCA, and Linear Discriminant Analysis (LDA). This includes providing proofs for discriminant directions, applying the ”kernel trick” to nonlinear datasets like concentric circles, and performing manual calculations for Fisher’s criterion.

The final section evaluates different feature selection techniques, comparing filter, wrapper, and embedded methods like Sequential Forward Selection and Lasso. Additionally, image segmentation is explored by applying K-Means and Gaussian Mixture Models (GMM) to 5-dimensional feature vectors to classify pixels based on their spatial and color information.

# Contents

## 1. One Full Iteration of K-Means

Consider the dataset $D$ consisting of five points. You are asked to perform one full iteration of k-means clustering on this dataset with $k = 2$, using Euclidean distance. The data points are:

- $p_1 = (5.5, 3.1)$
- $p_2 = (5.1, 4.8)$
- $p_3 = (6.6, 3.0)$
- $p_4 = (5.5, 4.6)$
- $p_5 = (6.8, 3.8)$

The initial cluster centers (centroids) are:

- $\mu_0 = (5.3, 3.5)$
- $\mu_1 = (5.1, 4.2)$

(a) Assign each data point to the nearest cluster center.

(b) Compute the updated cluster centers as the mean of the points assigned to each cluster.

(c) Report the new center for Cluster 0 and the new center for Cluster 1.

(d) State the final cardinality (number of points) of Cluster 0 after the update.

### **1.1** Step 1: Assign Data Points to Nearest Cluster

We assign each point to the cluster with the nearest centroid based on Euclidean distance. 1. Point $p_1 = (5.5, 3.1)$:

$$d(p_1, \mu_0)^2 = (5.5 - 5.3)^2 + (3.1 - 3.5)^2 = (0.2)^2 + (-0.4)^2 = 0.04 + 0.16 = 0.20$$
$$d(p_1, \mu_1)^2 = (5.5 - 5.1)^2 + (3.1 - 4.2)^2 = (0.4)^2 + (-1.1)^2 = 0.16 + 1.21 = 1.37$$

$0.20 < 1.37$, assign $p_1 \rightarrow$ Cluster 0.

2. Point $p_2 = (5.1, 4.8)$:

$$d(p_2, \mu_0)^2 = (5.1 - 5.3)^2 + (4.8 - 3.5)^2 = (-0.2)^2 + (1.3)^2 = 0.04 + 1.69 = 1.73$$
$$d(p_2, \mu_1)^2 = (5.1 - 5.1)^2 + (4.8 - 4.2)^2 = (0)^2 + (0.6)^2 = 0.36$$

$0.36 < 1.73$, assign $p_2 \rightarrow$ Cluster 1.

3. Point $p_3 = (6.6, 3.0)$:

$$d(p_3, \mu_0)^2 = (6.6 - 5.3)^2 + (3.0 - 3.5)^2 = (1.3)^2 + (-0.5)^2 = 1.69 + 0.25 = 1.94$$
$$d(p_3, \mu_1)^2 = (6.6 - 5.1)^2 + (3.0 - 4.2)^2 = (1.5)^2 + (-1.2)^2 = 2.25 + 1.44 = 3.69$$

$1.94 < 3.69$, assign $p_3 \to$ Cluster 0.

**4. Point $p_4 = (5.5, 4.6)$:**

$$d(p_4, \mu_0)^2 = (5.5 - 5.3)^2 + (4.6 - 3.5)^2 = (0.2)^2 + (1.1)^2 = 0.04 + 1.21 = 1.25$$
$$d(p_4, \mu_1)^2 = (5.5 - 5.1)^2 + (4.6 - 4.2)^2 = (0.4)^2 + (0.4)^2 = 0.16 + 0.16 = 0.32$$

$0.32 < 1.25$, assign $p_4 \to$ Cluster 1.

5. Point $p_5 = (6.8, 3.8)$:

$$d(p_5, \mu_0)^2 = (6.8 - 5.3)^2 + (3.8 - 3.5)^2 = (1.5)^2 + (0.3)^2 = 2.25 + 0.09 = 2.34$$
$$d(p_5, \mu_1)^2 = (6.8 - 5.1)^2 + (3.8 - 4.2)^2 = (1.7)^2 + (-0.4)^2 = 2.89 + 0.16 = 3.05$$

Since $2.34 < 3.05$, assign $p_5 \to$ Cluster 0.

## 1.2 Step 2: Compute Updated Cluster Centers

The new cluster centers are computed as the mean vector of the points assigned to each cluster.

$\mu_0'$:

$$\mu_0' = \frac{1}{|C_0|} \sum_{p \in C_0} p = \frac{p_1 + p_3 + p_5}{3}$$
$$x_{new} = \frac{5.5 + 6.6 + 6.8}{3} = \frac{18.9}{3} = 6.3$$
$$y_{new} = \frac{3.1 + 3.0 + 3.8}{3} = \frac{9.9}{3} = 3.3$$
$$\Rightarrow \mu_0' = (6.3, 3.3)$$

$\mu_1'$:

$$\mu_1' = \frac{1}{|C_1|} \sum_{p \in C_1} p = \frac{p_2 + p_4}{2}$$
$$x_{new} = \frac{5.1 + 5.5}{2} = \frac{10.6}{2} = 5.3$$
$$y_{new} = \frac{4.8 + 4.6}{2} = \frac{9.4}{2} = 4.7$$
$$\Rightarrow \mu_1' = (5.3, 4.7)$$

### 1.3 Final Results

New Center for Cluster 0: $(6.3, 3.3)$
New Center for Cluster 1: $(5.3, 4.7)$
Final Cardinality of Cluster 0: The number of points in Cluster 0 is **3**.

## 2. Linkage Strategies in Agglomerative Clustering

In bottom-up agglomerative clustering, two common strategies for determining the distance between clusters are single linkage and complete linkage, which consider the minimum and maximum distance between cluster members as the merger criterion, respectively.

(a) Compare the computational complexity of these two methods.

(b) Which method is more robust to outliers? Justify your answer.

(c) Using the complete linkage method, perform clustering on the following dataset and then draw the resulting dendrogram.

### 2.1 1. Computational Complexity Comparison

Both Single Linkage and Complete Linkage share the same underlying agglomerative hierarchical clustering framework, leading to similar computational complexities.

- **Standard Algorithm:** For a dataset of $N$ points, both methods typically require calculating the initial pairwise distance matrix, which takes $O(N^2)$. There are $N-1$ iterations. In a naive implementation, finding the closest pair of clusters takes $O(N^2)$ per iteration, and updating the distance matrix takes $O(N)$, resulting in a total complexity of $O(N^3)$.
  - **Priority Queue Implementation ($O(N^2 \log N)$):** This is the general-purpose approach used in most standard libraries. It maintains all pairwise distances in a Min-Heap (Priority Queue) to quickly retrieve the closest pair in $O(1)$. However, when two clusters merge, we must calculate their new distances to all other clusters and update the heap. Since each heap insertion or update takes $O(\log N)$, and we perform this for roughly $N$ neighbors over $N$ iterations, the total complexity becomes $O(N^2 \log N)$.
  - **Single Linkage Optimization ($O(N^2)$):** We can optimize this further by exploiting the graph-theory property that Single Linkage clustering is equivalent to finding a Minimum Spanning Tree (MST). specialized algorithms like SLINK allow us to

update the nearest-neighbor distances using simple array operations without the logarithmic overhead of a heap, reducing the complexity to a strict $O(N^2)$.

– **Complete Linkage Optimization ($O(N^2)$):** Similarly, Complete Linkage can be optimized to $O(N^2)$ using the CLINK algorithm. Although it has the same asymptotic complexity as Single Linkage, it is practically slower because it requires more complex pointer updates to track the *maximum* diameter of the merged clusters, rather than just the minimum edge.

## 2.2 2. Robustness to Outliers

Complete Linkage is more robust to outliers than Single Linkage.

- **Single Linkage:** This method defines distance based on the closest pair of points, which makes it very sensitive to noise. If there are a few outlier points sitting between two separate clusters, the algorithm treats them like a ”bridge” and merges the two groups together. This is called the chaining effect, and it tends to create long, stringy clusters rather than distinct ones.
- **Complete Linkage:** This method defines distance based on the farthest pair of points, making it much stricter. Because it looks at the ”worst-case” distance, a single outlier cannot easily bridge two clusters unless it is close to the entire group. This effectively allows the algorithm to ignore bridging noise and produce tighter, more spherical clusters.

## 2.3 3. Complete Linkage Clustering on Dataset

### 2.3.1 Dataset Coordinates

Based on the visual grid provided in the problem, the coordinates for the points $p_1$ through $p_9$ are:

- $p_1 = (1, 1)$ $p_2 = (3, 2)$ $p_3 = (9, 1)$
- $p_4 = (3, 7)$ $p_5 = (7, 2)$ $p_6 = (9, 7)$
- $p_7 = (4, 8)$ $p_8 = (8, 3)$ $p_9 = (1, 4)$

### 2.3.2 Clustering Steps (Complete Linkage)

We calculate the squared Euclidean distances ($d^2$) to avoid roots until necessary. We iteratively merge the pair with the smallest maximum distance.

**Step 1: Initial Closest Pairs**

- $d^2(p_4, p_7) = (4-3)^2 + (8-7)^2 = 1 + 1 = 2$.

- $d^2(p_5, p_8) = (8-7)^2 + (3-2)^2 = 1 + 1 = 2$.

Merge $C_1 = \{p_4, p_7\}$ and $C_2 = \{p_5, p_8\}$.

**Step 2: Next Closest Pairs**

- $d^2(p_1, p_2) = (3-1)^2 + (2-1)^2 = 4 + 1 = 5$.
- Distance from $p_3$ to $C_2$: $\max(d^2(p_3, p_5), d^2(p_3, p_8))$. $d^2(p_3, p_5) = (7-9)^2 + (2-1)^2 = 5$. $d^2(p_3, p_8) = (8-9)^2 + (3-1)^2 = 5$. Max is 5.

Merge $C_3 = \{p_1, p_2\}$. Merge $p_3$ into $C_2 \to C_{Right} = \{p_3, p_5, p_8\}$.

**Step 3: Merging** $p_9$

- Distance $p_9$ to $C_3\{p_1, p_2\}$: $d^2(p_9, p_1) = (1-1)^2 + (4-1)^2 = 9$. $d^2(p_9, p_2) = (1-3)^2 + (4-2)^2 = 8$. Max is 9.
- This is the minimal active distance (compared to others like $p_6$ to clusters).

Merge $p_9$ into $C_3 \to C_{Left} = \{p_1, p_2, p_9\}$.

**Step 4: Merging** $p_6$

- Distance $p_6$ to $C_{Right}\{p_3, p_5, p_8\}$: Max distance is to $p_3(9, 1)$. $d^2(p_6, p_3) = (9-9)^2 + (7-1)^2 = 36$.
- Distance $p_6$ to $C_1\{p_4, p_7\}$: Max distance is to $p_4(3, 7)$. $d^2(p_6, p_4) = (9-3)^2 + (7-7)^2 = 36$.
- (Tie Break): We merge $p_6$ with $C_{Right}$ based on closer proximity to the centroid/other members (e.g., $p_8$).

Merge $p_6$ into $C_{Right} \to C_{R\_Final} = \{p_3, p_5, p_8, p_6\}$.

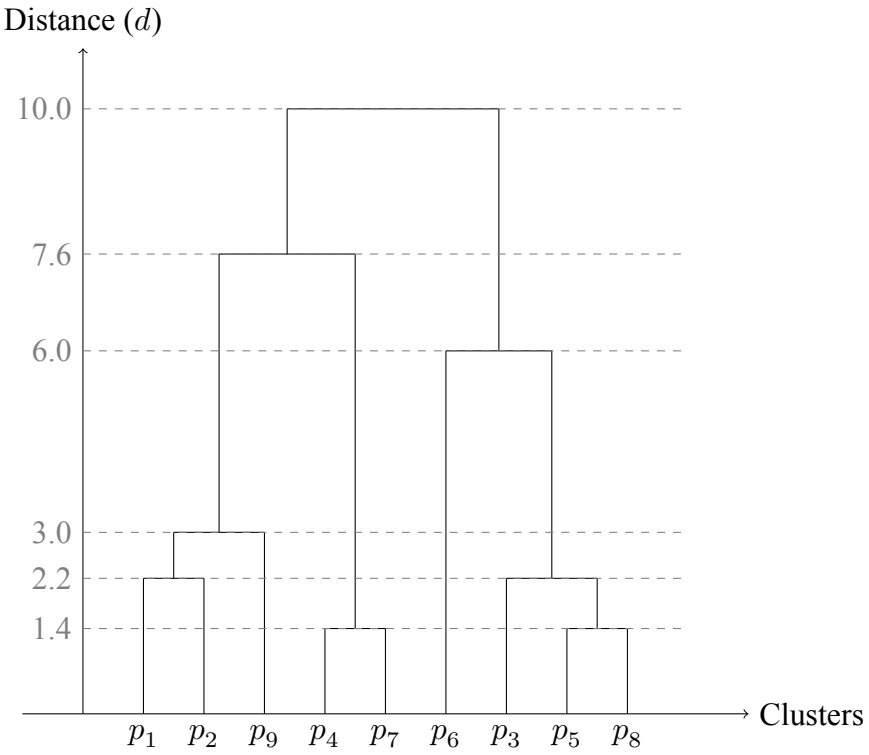
**Step 5: Merging Clusters** We have three clusters: $C_{Left}\{p_1, p_2, p_9\}$, $C_{Top}\{p_4, p_7\}$, and $C_{R\_Final}$.

- $d_{max}(C_{Left}, C_{Top})$: Max is $d(p_1, p_7) \approx \sqrt{58} \approx 7.6$.
- $d_{max}(C_{Left}, C_{R\_Final})$: Max is $d(p_1, p_6) = \sqrt{100} = 10$.
- $d_{max}(C_{Top}, C_{R\_Final})$: Max is $d(p_7, p_3) \approx \sqrt{74} \approx 8.6$.

Merge $C_{Left}$ and $C_{Top}$ at distance $\approx 7.6$. New Cluster $C_{Main} = \{p_1, p_2, p_9, p_4, p_7\}$.

**Step 6: Final Merge** Merge $C_{Main}$ and $C_{R\_Final}$ at distance 10 (defined by $p_1$ and $p_6$).

### 2.3.3 Resulting Dendrogram




## 3. Dimensionality Reduction and Data Geometry

Consider several datasets with different geometric and statistical properties. These include linearly correlated Gaussian data, two concentric circles, high-dimensional data with strong class overlap, and data containing many irrelevant or noisy features.
For each case, determine whether PCA, Kernel PCA, or LDA is the most appropriate dimensionality reduction method.
Justify your choice by referring to the objective function of the method, the geometry of the data, and the assumptions underlying each technique. Your explanation should clearly demonstrate an understanding of why certain methods succeed or fail depending on the data distribution.

**3.1** Case 1: Linearly Correlated Gaussian Data

**PCA**

- Geometry & Assumptions: The data distribution is described as ”linearly correlated” and ”Gaussian.” A multidimensional Gaussian distribution forms a hyper-ellipsoid shape in the input space. PCA assumes that the relevant structure of the data lies in a linear

subspace and that the axes of maximum variance correspond to the most informative directions.

- Objective Function: PCA seeks to find orthogonal directions (principal components) that maximize the variance of the projected data.
- Justification: Since the data is Gaussian, its structure is fully characterized by its mean vector and covariance matrix. PCA diagonalizes the covariance matrix, aligning the principal components exactly with the axes of the Gaussian ellipsoid. Since the correlations are linear, a linear projection is sufficient to capture the data's geometry without the need for kernels or class labels.

## **3.2** Case 2: Two Concentric Circles

Kernel PCA

- Geometry & Assumptions: The data consists of two concentric circles. This geometry is inherently **nonlinear**; the classes cannot be separated or unfolded by a single straight line or linear plane in the original space.
- Objective Function: Kernel PCA extends standard PCA to a high-dimensional feature space using the "kernel trick." It diagonalizes the covariance matrix in this feature space to find nonlinear principal components.
- Justification: Standard PCA and LDA are linear methods and would fail to capture the circular structure (projecting concentric circles onto a line results in overlapping segments). Kernel PCA, particularly with a Radial Basis Function (RBF) kernel, implicitly maps the data into a higher-dimensional space where the distance from the origin (radius) becomes a linear dimension. In this lifted space, the concentric circles can be linearly separated or represented as distinct clusters.

## **3.3** Case 3: High-Dimensional Data with Strong Class Overlap

LDA

- Geometry & Assumptions: The data is high-dimensional with "strong class overlap". The explicit mention of "classes" implies a supervised learning context. "Overlap" suggests that the means of the classes may be close relative to their variance in many directions.
- Objective Function: LDA aims to maximize the Fisher criterion: $J(w) = \frac{w^T S_B w}{w^T S_W w}$, maximizing the distance between class means ($S_B$) while minimizing the scatter within each class ($S_W$).

- Justification: PCA is unsupervised and maximizes total variance. If the direction of maximum variance is caused by noise or shared features that do not distinguish the classes, PCA will preserve this overlap. LDA, however, explicitly uses class labels to find the specific subspace that discriminates between the classes. Even with strong overlap, LDA is the only method among the three that actively attempts to minimize the within-class scatter to improve separability.

### **3.4** Case 4: Data Containing Many Irrelevant or Noisy Features

PCA

- Geometry & Assumptions: The dataset contains many ”irrelevant” features, which generally implies dimensions that do not contain structured signal and are uncorrelated with the underlying data manifold (often modeled as low-variance isotropic noise).
- Objective Function: PCA minimizes the reconstruction error, which is equivalent to maximizing the variance retained in the top $k$ components.
- Justification: PCA is widely used for noise reduction and feature extraction. By selecting only the top $k$ principal components (associated with the largest eigenvalues), PCA reconstructs the data using the directions of the strongest signal. The ”irrelevant” or noisy features typically correspond to the directions with the smallest eigenvalues (the ”tail” of the spectrum). Dropping these components effectively filters out the noise and irrelevant dimensions, compressing the data into its intrinsic dimensionality.

## 4. Formal Theory of LDA and Kernel PCA

**Part 1:** Prove that for a classification problem with $C$ classes, Linear Discriminant Analysis can produce at most $C - 1$ nonzero discriminant directions. Your proof should rely on rank arguments involving the between-class scatter matrix.
**Part 2:** Then consider Kernel PCA. Starting from the covariance operator in feature space, explain why direct eigen-decomposition is infeasible. Show that every principal component in feature space can be written as a linear combination of mapped training samples. Derive the kernel eigenvalue problem and obtain the expression for projecting a new data point using only kernel evaluations.

### **4.1** Proof of LDA Rank (At most $C - 1$)

The objective of LDA is to find projection directions $w$ that maximize the Fisher criterion. These optimal directions are the eigenvectors corresponding to the non-zero eigenvalues of the

matrix $S_W^{-1}S_B$, where $S_B$ is the between-class scatter matrix and $S_W$ is the within-class scatter matrix.

The rank of the product matrix $S_W^{-1}S_B$ is limited by the rank of its factors:

$$\text{rank}(S_W^{-1}S_B) \leq \min(\text{rank}(S_W^{-1}), \text{rank}(S_B)) = \text{rank}(S_B)$$

the number of non-zero eigenvalues (and discriminant directions) is bounded by $\text{rank}(S_B)$.

The between-class scatter matrix is defined as:

$$S_B = \sum_{c=1}^{C} N_c(\mu_c - \mu)(\mu_c - \mu)^T$$

$S_B$ is a sum of $C$ matrices, each of rank 1 (formed by the outer product of the vector $v_c = \mu_c - \mu$). Therefore, $S_B$ is the sum of $C$ vectors from the subspace spanned by $\{\mu_1 - \mu, \dots, \mu_C - \mu\}$. This implies:

$$\text{rank}(S_B) \leq C$$

However, the vectors $(\mu_c - \mu)$ are linearly dependent. We know that the global mean $\mu$ is the weighted average of the class means:

$$\mu = \frac{1}{N}\sum_{c=1}^{C} N_c\mu_c \implies \sum_{c=1}^{C} N_c\mu_c = N\mu = \sum_{c=1}^{C} N_c\mu$$

$$\sum_{c=1}^{C} N_c(\mu_c - \mu) = 0$$

Because the weighted sum of the vectors equals the zero vector, any one vector can be written as a linear combination of the other $C-1$ vectors. This linear dependence reduces the maximum possible rank of the subspace by 1.

**Conclusion:**

$$\text{rank}(S_B) \leq C - 1$$

Therefore, there are at most $C-1$ non-zero eigenvalues, and LDA can produce at most $C-1$ discriminant directions.

## **4.2** Kernel PCA Derivation

### 4.2.1 Infeasibility of Direct Eigen-decomposition

In Kernel PCA, The covariance matrix in the feature space is given by:

$$C = \frac{1}{N}\sum_{i=1}^{N} \Phi(x_i)\Phi(x_i)^T$$

If the dimensionality of $\mathcal{F}$ is very large or infinite (as with the Gaussian RBF kernel), the covariance matrix $C$ becomes an infinite-dimensional operator. Computing the eigenvalues and eigenvectors of an infinite-dimensional matrix directly is computationally infeasible or impossible.

### 4.2.2 Principal Components as Linear Combinations

We aim to solve the eigenvalue problem $C\mathbf{v} = \lambda\mathbf{v}$ for $\lambda > 0$ and $\mathbf{v} \in \mathcal{F}$. Substituting the definition of $C$:

$$\left(\frac{1}{N}\sum_{i=1}^{N}\Phi(x_i)\Phi(x_i)^T\right)\mathbf{v} = \lambda\mathbf{v}$$

$$\frac{1}{N}\sum_{i=1}^{N}\Phi(x_i)\left(\Phi(x_i)^T\mathbf{v}\right) = \lambda\mathbf{v}$$

The term $\Phi(x_i)^T\mathbf{v}$ is a scalar (a dot product). The equation shows that $\lambda\mathbf{v}$ is a weighted sum of the vectors $\Phi(x_i)$. Dividing by $\lambda$ (since $\lambda > 0$):

$$\mathbf{v} = \sum_{i=1}^{N}\left(\frac{\Phi(x_i)^T\mathbf{v}}{N\lambda}\right)\Phi(x_i) = \sum_{i=1}^{N}\alpha_i\Phi(x_i)$$

This proves that every principal component $\mathbf{v}$ lies in the span of the mapped training samples $\Phi(x_1), \ldots, \Phi(x_N)$.

### 4.2.3 Derivation of the Kernel Eigenvalue Problem

Substitute the expansion $\mathbf{v} = \sum_{j=1}^{N}\alpha_j\Phi(x_j)$ into $C\mathbf{v} = \lambda\mathbf{v}$:

$$\frac{1}{N}\sum_{i=1}^{N}\Phi(x_i)\Phi(x_i)^T\left(\sum_{j=1}^{N}\alpha_j\Phi(x_j)\right) = \lambda\sum_{j=1}^{N}\alpha_j\Phi(x_j)$$

$$\frac{1}{N}\sum_{i=1}^{N}\Phi(x_k)^T\Phi(x_i)\left(\sum_{j=1}^{N}\alpha_j\Phi(x_i)^T\Phi(x_j)\right) = \lambda\sum_{j=1}^{N}\alpha_j\Phi(x_k)^T\Phi(x_j)$$

We define the Kernel Matrix $K$ where $K_{ij} = \Phi(x_i)^T\Phi(x_j) = k(x_i, x_j)$. The equation simplifies in matrix notation:

$$\frac{1}{N}K^2\alpha = \lambda K\alpha$$

Assuming $K$ is invertible (or considering the relevant subspace), we multiply by $K^{-1}$:

$$K\alpha = N\lambda\alpha$$

This is the standard eigenvalue problem for the Kernel matrix $K$. We solve this to find the coefficient vectors $\alpha$.

### 4.2.4 Projection of a New Data Point

Once we have the coefficients $\alpha$ for an eigenvector $\mathbf{v}$, we can project a new test point $x_{new}$ onto this principal component without explicitly computing $\Phi(x_{new})$. The projection is the dot product in feature space:

$$y = \mathbf{v}^T\Phi(x_{new})$$

Substitute $\mathbf{v} = \sum_{i=1}^{N}\alpha_i\Phi(x_i)$:

$$y = \left(\sum_{i=1}^{N}\alpha_i\Phi(x_i)\right)^T\Phi(x_{new}) = \sum_{i=1}^{N}\alpha_i\left(\Phi(x_i)^T\Phi(x_{new})\right)$$

Using the kernel function:

$$y = \sum_{i=1}^{N} \alpha_i k(x_i, x_{new})$$

This allows us to perform dimensionality reduction entirely though kernel evaluations.

## 5. Manual Computation of LDA

Apply Linear Discriminant Analysis by hand to the following synthetic dataset consisting of two classes:

- Class 1: (4,2), (2, 4), (2, 3), (3, 6), (4, 4)
- Class 2: (9,10), (6,8), (9,5), (8,7), (10,8)

(a) Compute the sample mean of each class and the overall mean.

(b) Derive the within-class scatter matrix and the between-class scatter matrix.

(c) Solve the generalized eigenvalue problem associated with Fisher's criterion and determine the optimal discriminant direction.

(d) Finally, project all data points onto this direction and report their one-dimensional coordinates.

### **5.1** Sample Means

First, we compute the mean vector for each class and the global mean. Let $N_1 = 5$ and $N_2 = 5$.

$\mu_1$:

$$\mu_1 = \frac{1}{5} \sum_{x \in C_1} x = \frac{1}{5} \begin{pmatrix} 4+2+2+3+4 \\ 2+4+3+6+4 \end{pmatrix} = \frac{1}{5} \begin{pmatrix} 15 \\ 19 \end{pmatrix} = \begin{pmatrix} 3.0 \\ 3.8 \end{pmatrix}$$

$\mu_2$:

$$\mu_2 = \frac{1}{5} \sum_{x \in C_2} x = \frac{1}{5} \begin{pmatrix} 9+6+9+8+10 \\ 10+8+5+7+8 \end{pmatrix} = \frac{1}{5} \begin{pmatrix} 42 \\ 38 \end{pmatrix} = \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix}$$

Overall mean ($\mu$):

$$\mu = \frac{N_1\mu_1 + N_2\mu_2}{N_1 + N_2} = \frac{5 \begin{pmatrix} 3.0 \\ 3.8 \end{pmatrix} + 5 \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix}}{10} = \begin{pmatrix} 5.7 \\ 5.7 \end{pmatrix}$$

## **5.2** Scatter Matrices

$S_W$**:** $S_W = S_1 + S_2$, where $S_i = \sum_{x \in C_i} (x - \mu_i)(x - \mu_i)^T$.

For Class 1 ($C_1$):

$$\begin{aligned}
(4,2) - (3,3.8) = (1,-1.8) &\rightarrow \left(\begin{smallmatrix} 1 & -1.8 \\ -1.8 & 3.24 \end{smallmatrix}\right) \\
(2,4) - (3,3.8) = (-1,0.2) &\rightarrow \left(\begin{smallmatrix} 1 & -0.2 \\ -0.2 & 0.04 \end{smallmatrix}\right) \\
(2,3) - (3,3.8) = (-1,-0.8) &\rightarrow \left(\begin{smallmatrix} 1 & 0.8 \\ 0.8 & 0.64 \end{smallmatrix}\right) \\
(3,6) - (3,3.8) = (0,2.2) &\rightarrow \left(\begin{smallmatrix} 0 & 0 \\ 0 & 4.84 \end{smallmatrix}\right) \\
(4,4) - (3,3.8) = (1,0.2) &\rightarrow \left(\begin{smallmatrix} 1 & 0.2 \\ 0.2 & 0.04 \end{smallmatrix}\right)
\end{aligned}$$

Summing these gives $S_1$:

$$S_1 = \begin{pmatrix} 4 & -1 \\ -1 & 8.8 \end{pmatrix}$$

For Class 2 ($C_2$):

$$\begin{aligned}
(9,10) - (8.4,7.6) = (0.6,2.4) &\rightarrow \left(\begin{smallmatrix} 0.36 & 1.44 \\ 1.44 & 5.76 \end{smallmatrix}\right) \\
(6,8) - (8.4,7.6) = (-2.4,0.4) &\rightarrow \left(\begin{smallmatrix} 5.76 & -0.96 \\ -0.96 & 0.16 \end{smallmatrix}\right) \\
(9,5) - (8.4,7.6) = (0.6,-2.6) &\rightarrow \left(\begin{smallmatrix} 0.36 & -1.56 \\ -1.56 & 6.76 \end{smallmatrix}\right) \\
(8,7) - (8.4,7.6) = (-0.4,-0.6) &\rightarrow \left(\begin{smallmatrix} 0.16 & 0.24 \\ 0.24 & 0.36 \end{smallmatrix}\right) \\
(10,8) - (8.4,7.6) = (1.6,0.4) &\rightarrow \left(\begin{smallmatrix} 2.56 & 0.64 \\ 0.64 & 0.16 \end{smallmatrix}\right)
\end{aligned}$$

Summing these gives $S_2$:

$$S_2 = \begin{pmatrix} 9.2 & -0.2 \\ -0.2 & 13.2 \end{pmatrix}$$

Total Within-Class Scatter ($S_W$):

$$S_W = S_1 + S_2 = \begin{pmatrix} 4 + 9.2 & -1 - 0.2 \\ -1 - 0.2 & 8.8 + 13.2 \end{pmatrix} = \begin{pmatrix} 13.2 & -1.2 \\ -1.2 & 22.0 \end{pmatrix}$$

Between-Class Scatter Matrix:

$$S_B = \sum_{c=1}^{2} N_c (\mu_c - \mu)(\mu_c - \mu)^T$$

Calculating the difference vectors: $\mu_1 - \mu = (-2.7, -1.9)$ and $\mu_2 - \mu = (2.7, 1.9)$. Since $N_1 = N_2 = 5$, and the vectors are symmetric opposites:

$$S_B = 5 \begin{pmatrix} (-2.7)^2 & (-2.7)(-1.9) \\ (-2.7)(-1.9) & (-1.9)^2 \end{pmatrix} + 5 \begin{pmatrix} (2.7)^2 & (2.7)(1.9) \\ (2.7)(1.9) & (1.9)^2 \end{pmatrix}$$

$$S_B = 10 \begin{pmatrix} 7.29 & 5.13 \\ 5.13 & 3.61 \end{pmatrix} = \begin{pmatrix} 72.9 & 51.3 \\ 51.3 & 36.1 \end{pmatrix}$$

## **5.3** Optimal Discriminant Direction

We solve for the direction $w$ that maximizes $J(w) = \frac{w^T S_B w}{w^T S_W w}$. For a two-class problem, the optimal direction is given by:

$$w \propto S_W^{-1}(\mu_1 - \mu_2)$$

First, compute $S_W^{-1}$:

$$\det(S_W) = (13.2)(22.0) - (-1.2)(-1.2) = 290.4 - 1.44 = 288.96$$

$$S_W^{-1} = \frac{1}{288.96}\begin{pmatrix} 22.0 & 1.2 \\ 1.2 & 13.2 \end{pmatrix}$$

Difference in means:

$$\mu_1 - \mu_2 = \begin{pmatrix} 3.0 \\ 3.8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} = \begin{pmatrix} -5.4 \\ -3.8 \end{pmatrix}$$

Calculate $w$ (ignoring the positive scalar factor $\frac{1}{288.96}$ as it does not affect direction):

$$w \propto \begin{pmatrix} 22.0 & 1.2 \\ 1.2 & 13.2 \end{pmatrix}\begin{pmatrix} -5.4 \\ -3.8 \end{pmatrix}$$

$$w_1 = (22.0)(-5.4) + (1.2)(-3.8) = -118.8 - 4.56 = -123.36$$

$$w_2 = (1.2)(-5.4) + (13.2)(-3.8) = -6.48 - 50.16 = -56.64$$

$$w = \begin{pmatrix} -123.36 \\ -56.64 \end{pmatrix}$$

For simpler reporting, we can normalize this vector. The Euclidean norm is $\approx 135.75$. The normalized direction is:

$$w_{norm} \approx \begin{pmatrix} -0.9087 \\ -0.4172 \end{pmatrix}$$

## **5.4** Projection of Data Points

We project the points onto the 1D subspace defined by $w_{norm}$ using $y = w_{norm}^T x$.

| Class | Point $(x_1, x_2)$ | Calculation | Projected Value $y$ |
|---|---|---|---|
| **Class 1** | | | |
| 1 | (4, 2) | $-0.9087(4) - 0.4172(2)$ | $-4.47$ |
| 1 | (2, 4) | $-0.9087(2) - 0.4172(4)$ | $-3.49$ |
| 1 | (2, 3) | $-0.9087(2) - 0.4172(3)$ | $-3.07$ |
| 1 | (3, 6) | $-0.9087(3) - 0.4172(6)$ | $-5.23$ |
| 1 | (4, 4) | $-0.9087(4) - 0.4172(4)$ | $-5.30$ |
| **Class 2** | | | |
| 2 | (9, 10) | $-0.9087(9) - 0.4172(10)$ | $-12.35$ |
| 2 | (6, 8) | $-0.9087(6) - 0.4172(8)$ | $-8.79$ |
| 2 | (9, 5) | $-0.9087(9) - 0.4172(5)$ | $-10.26$ |
| 2 | (8, 7) | $-0.9087(8) - 0.4172(7)$ | $-10.19$ |
| 2 | (10, 8) | $-0.9087(10) - 0.4172(8)$ | $-12.42$ |

*Analysis:* The projected values for Class 1 range roughly from $-3.0$ to $-5.3$, while Class 2 ranges from $-8.7$ to $-12.4$. The classes are well-separated in the 1D space, shows the effectiveness of LDA discriminant direction.

## 6. Question 6: Feature Selection Strategies

**Topic: Feature Selection Methods (Filter, Wrapper, Embedded)**

- Explain in detail how Sequential Forward Selection (SFS) and Sequential Backward Selection (SBS) operate, including their initialization, update steps, and stopping criteria.
- Describe how bidirectional selection combines elements of both.
- Discuss why these methods are considered ”greedy” and what that term implies in this context.
- Analyze the consequences of this greedy behavior, specifically why they are not guaranteed to find the globally optimal subset even with a convex model. Provide a concrete example of suboptimality.
- Compare sequential selection with Recursive Feature Elimination (RFE) and L1 regularization (Lasso), addressing computational complexity, sensitivity to correlation, feature interactions, and suitability for high-dimensional data.

### **6.1** Sequential Selection Methods

#### 6.1.1 Sequential Forward Selection (SFS)

SFS is a bottom-up search strategy that builds a feature subset by adding one feature at a time.

- Initialization: Start with an empty feature set $S = \emptyset$.
- Update Step: In each iteration, train the model $d_{remaining}$ times, where each model uses the features in $S$ plus one candidate feature from the pool of unselected features.

$$f^* = \underset{f \notin S}{\text{argmax}}\, J(S \cup \{f\})$$

  The feature $f^*$ that yields the maximum improvement in the objective function $J$ (e.g., cross-validation accuracy) is permanently added to $S$.
- Stopping Criteria: The process stops when a desired number of features $k$ is reached, or when adding a new feature does not significantly improve the objective function (e.g., improvement $< \epsilon$).

### 6.1.2 Sequential Backward Selection (SBS)

SBS is a top-down search strategy that prunes features from the full set.

- Initialization: Start with the full set of all features $S = \{f_1, f_2, ..., f_d\}$.
- Update Step: In each iteration, train the model by temporarily removing one feature at a time.

$$f^* = \underset{f \in S}{\text{argmax}}\, J(S \quad \{f\})$$

  The feature $f^*$ whose removal results in the least decay (or highest retention) of performance is permanently removed from $S$.
- Stopping Criteria: Stops when the set is reduced to a target size $k$, or when removing further features causes a significant drop in performance.

### 6.1.3 Bidirectional (Stepwise) Selection

Bidirectional selection combines SFS and SBS to avoid the rigidity of purely monotonic growth or shrinkage.

- Operation: The algorithm alternates between forward and backward steps. For example, it might perform one forward step to add the best feature, followed by one backward step to check if any of the previously added features have become redundant given the new addition.
- Advantage: This allows the method to ”correct” previous decisions (e.g., removing a feature that was good individually but is now redundant in the presence of a new feature).

## 6.2 2. Analysis of Greedy Behavior

**Definition of ”Greedy”:** In this context, an algorithm is ”greedy” because it makes the locally optimal choice at each iteration with the hope that these choices will lead to a globally optimal

solution. It evaluates the benefit of a feature *conditioned only on the current subset*, without considering future iterations.

**Consequences and Suboptimality:** SFS and SBS suffer from the ”nesting effect”: once a feature is added (in SFS) or removed (in SBS), it cannot be re-evaluated (except in bidirectional methods).

- Failure Case (Feature Interactions): These methods may fail to capture interaction effects where features are weak individually but strong together (e.g., the XOR problem).
- Concrete Example: Consider a classification problem where Class 1 is defined by the XOR relationship of features $x_1$ and $x_2$, and Class 0 is random noise. Assume there is a third feature $x_3$ that is weakly correlated with the label on its own (e.g., accuracy 60%).
  - Individually, $x_1$ and $x_2$ yield 50% accuracy (random guess) because XOR requires both.
  - SFS will evaluate $\{x_1\}$, $\{x_2\}$, and $\{x_3\}$. It picks $\{x_3\}$ (60%).
  - In the next step, it adds the best remaining feature. Even if it adds $x_1$, the subset $\{x_3, x_1\}$ might be inferior to the optimal subset $\{x_1, x_2\}$ (100%).
  - Result: SFS selects a suboptimal subset starting with $x_3$, missing the critical $\{x_1, x_2\}$ pair because neither looked ”best” at step 1.

## **6.3** 3. Comparison with RFE and Lasso

| Criterion | Sequential Selection (SFS/SBS) | Recursive Feature Elimination (RFE) | L1 Regularization (Lasso) |
|---|---|---|---|
| Computational Complexity | High. requires retraining the model $O(d^2)$ times (where $d$ is feature count). | Medium. Requires training one model per elimination step, but often eliminates chunks of features. | Low. Feature selection happens during a single training process (optimization). |
| Feature Correlations | Can handle redundancy if the metric (e.g., accuracy) reflects it, but prone to multicollinearity instability. | Handles correlations relatively well by re-evaluating weights after every removal. | Tends to arbitrarily select one feature from a group of highly correlated features and zero out the others. |
| Feature Interactions | Bad. Misses interactions unless features are selected early. | Better. Multivariate model weights can capture interactions before elimination begins. | Limited. Linear models (standard Lasso) cannot capture non-linear interactions without explicit feature engineering. |
| High-Dimensional Data | Unsuitable. Computationally prohibitive for very large $d$. | Suitable. Efficient enough for moderate-to-high dimensions. | Highly Suitable. efficient optimization, works well even when $d > N$. |

# 7. Question 7: PCA and Kernel PCA on Structured Data

## 7.1 Part (a): PCA on the Iris Dataset

Using the standardized Iris dataset, we computed the eigenvalues and explained variance ratios for the first two principal components:

- **Eigenvalues:** $\lambda_1 \approx 2.94$, $\lambda_2 \approx 0.92$
- Explained Variance Ratio: PC1 explains $72\%$ of the variance, and PC2 explains $22\%$. Together, they preserve $95\%$ of the total information.

**Analysis of Results**

- Dominant Component: The first principal component (PC1) captures the vast majority of the variance ($\approx 73\%$). This occurs because the features in the Iris dataset (likely petal length and width) are highly correlated, creating a dominant direction of variation along which the data spreads out the most.

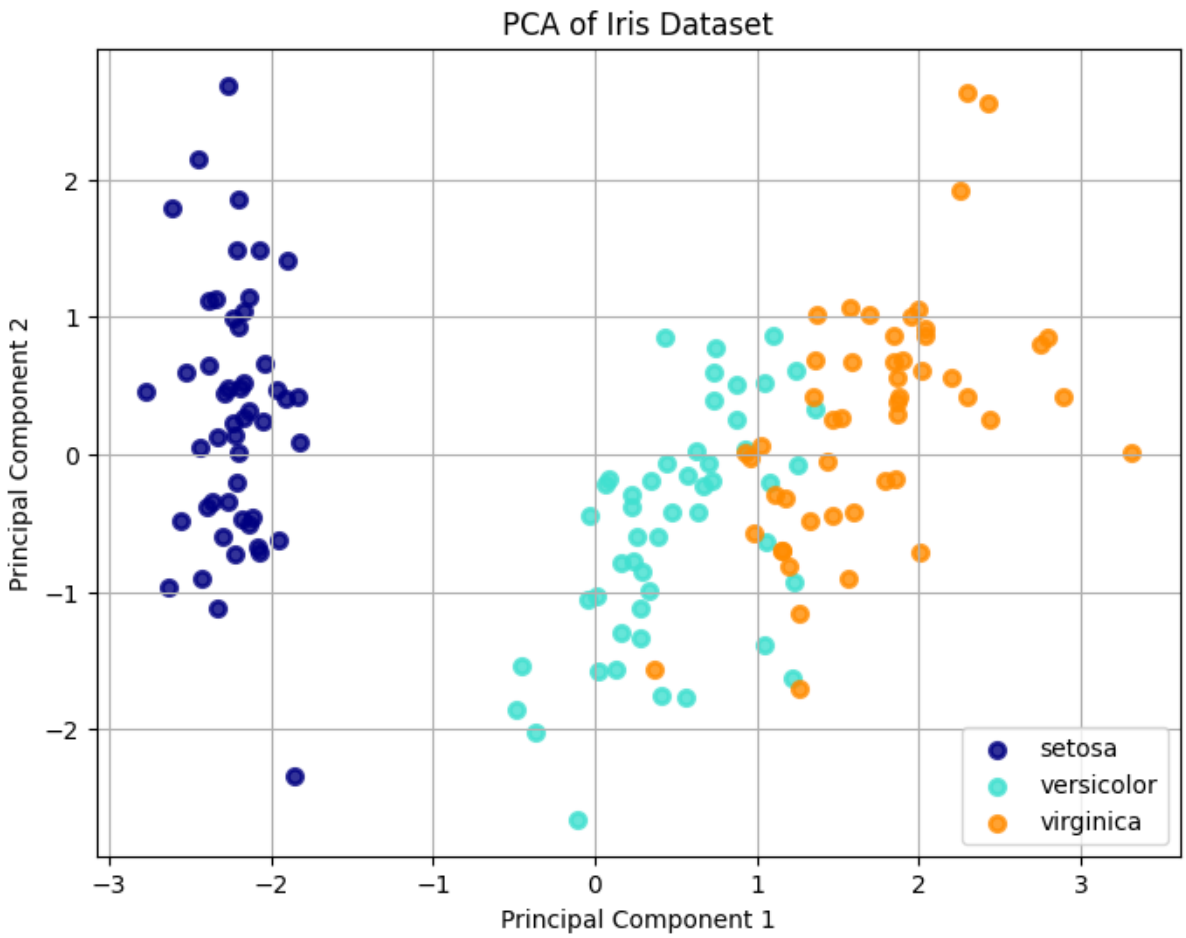



Figure 1: PCA projection of the Iris dataset onto the first two principal components. Setosa (dark blue) is well-separated, while Versicolor (cyan) and Virginica (orange) show some overlap.

- Class Separability: In the reduced space, the class *setosa* is perfectly separated from the others. However, *versicolor* and virginica exhibit some overlap. This is expected because PCA is an unsupervised technique. Its objective function maximizes the total variance of the projection (Var$(u^T x)$) without any knowledge of class labels. It preserves the global structure of the data, not necessarily the discrimination between specific classes.
- Appropriateness for Classification: While PCA reduces dimensionality effectively, it is not strictly optimal for classification. If the discriminative information between two classes lies in a direction of low variance (e.g., orthogonal to the main spread), PCA might discard it as ”noise.”
- Comparison with LDA: If Linear Discriminant Analysis (LDA) were used instead, the projection would differ significantly. LDA is supervised; its objective is to maximize the ratio of between-class scatter to within-class scatter ($J(w) = \frac{w^T S_B w}{w^T S_W w}$). LDA would specifically seek a projection direction that pushes the class means apart while minimizing the spread within each class, likely resulting in better separation between the overlapping versicolor and virginica clusters.

## 7.2 Part (b): Nonlinear Data and Kernel PCA

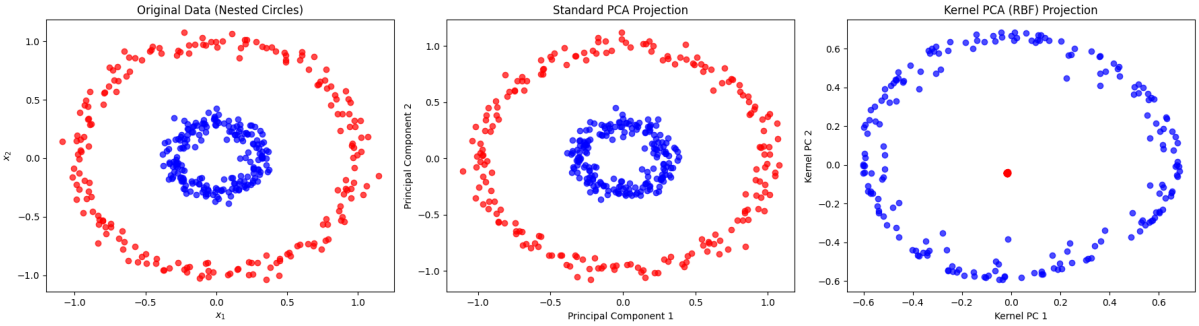



Figure 2: Comparison of Standard PCA (middle) and Kernel PCA with RBF kernel (right) on the Nested Circles dataset.

### 7.2.1 Analysis of Results

- Standard PCA Failure: As shown in the middle plot above, standard PCA fails to separate the two nested circles. Since PCA is a linear transformation (a rotation and projection of axes), it cannot fundamentally change the topology of the data. Linearly projecting concentric circles onto any 2D plane preserves their concentric nature, leaving them linearly inseparable.
- Kernel PCA Success: The rightmost plot shows the result of Kernel PCA using a Gaussian (RBF) kernel. Here, the two classes are clearly distinguished (the inner red circle is mapped to a tight central cluster, while the outer blue circle forms a distinct ring around it).
- Role of Nonlinear Mapping: The kernel trick allows us to implicitly map the input data $\mathbf{x}$ into a high-dimensional feature space $\Phi(\mathbf{x})$ where the data becomes linearly separable. The RBF kernel $k(\mathbf{x}, \mathbf{y}) = \exp(-\gamma||\mathbf{x} - \mathbf{y}||^2)$ measures similarity based on distance. In the feature space, the ”inner” points and ”outer” points are mapped to different regions (conceptually, the RBF kernel lifts the center points to a peak while the outer ring remains lower), allowing the principal components in that space to capture the radial structure as a primary feature.

# 8. Question 8: Feature Selection and Visualization on a Real Dataset

## 8.1 Exploratory Analysis and Correlation Filtering

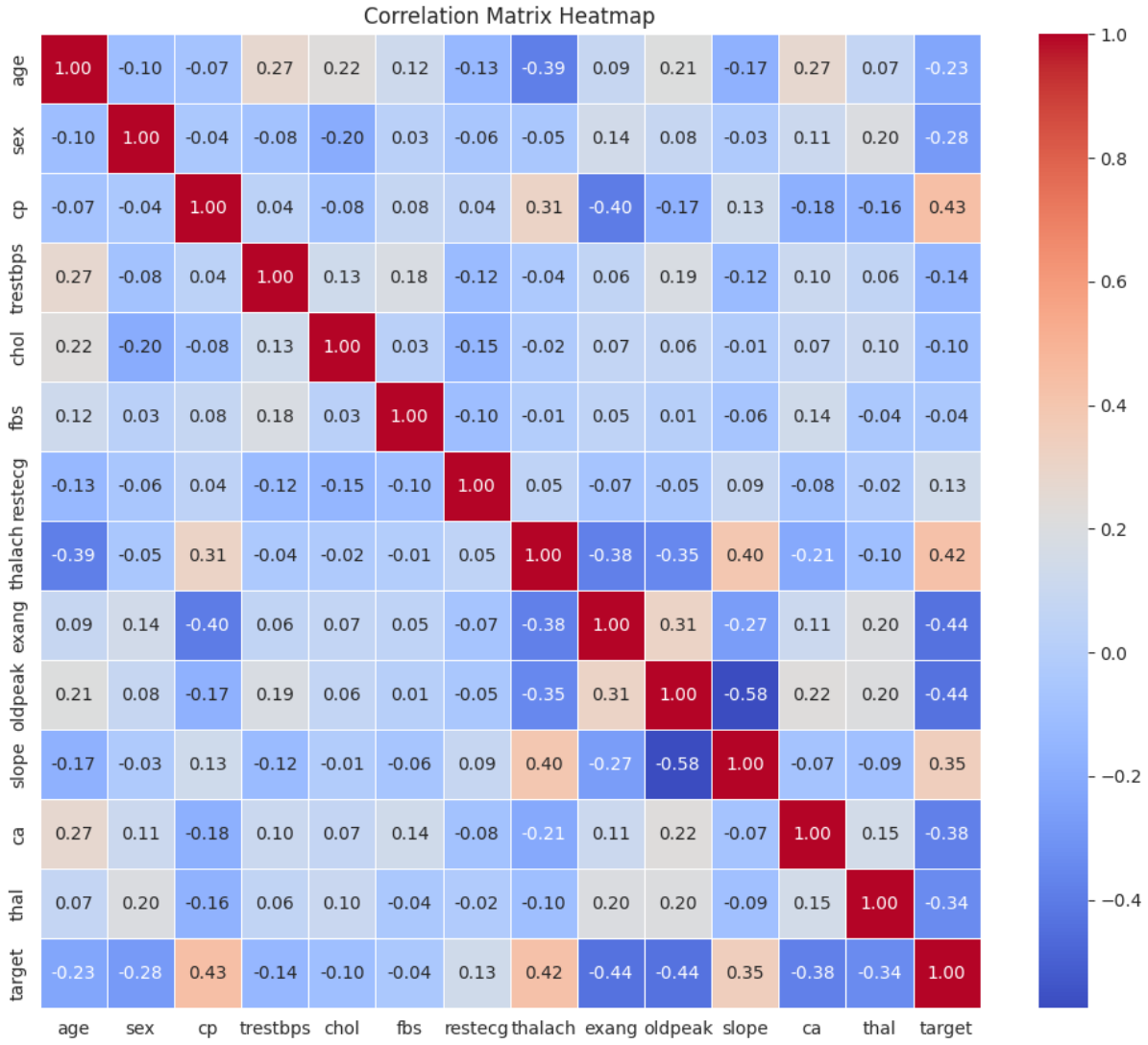



Figure 3: Correlation Matrix Heatmap of the Heart Disease dataset.

- The correlation matrix (Figure above) reveals some moderate relationships, such as a negative correlation between oldpeak and slope ($-0.58$) and a positive correlation between target and cp ($0.43$).
- Redundancy Removal: No features exhibited extremely high correlation (e.g., $> 0.90$) that would warrant immediate removal based purely on linear redundancy. Consequently, a strict correlation-based filter did not remove any features. This suggests that while features are related, they likely contain distinct information or the relationships are non-linear.

## **8.2** Univariate Selection vs. Recursive Feature Elimination (RFE)

We applied RFE (using a linear model) to select the top 5 features and compared them with a Univariate (Mutual Information) approach.

- **RFE Selected (Top 5):** ['cp', 'thalach', 'oldpeak', 'ca', 'thal']
- **Univariate Selected (Top 5):** ['oldpeak', 'cp', 'thalach', 'chol', 'thal']
- **Comparison:** There is a high degree of overlap (4 out of 5 features are common). This consistency reinforces that cp (chest pain), thalach (max heart rate), and oldpeak are strong, robust predictors regardless of the selection method.
- **Performance:** The model trained on the RFE subset achieved an accuracy of $\approx 78.5\%$.

## 8.3 Recursive Feature Elimination with Cross-Validation (RFECV)

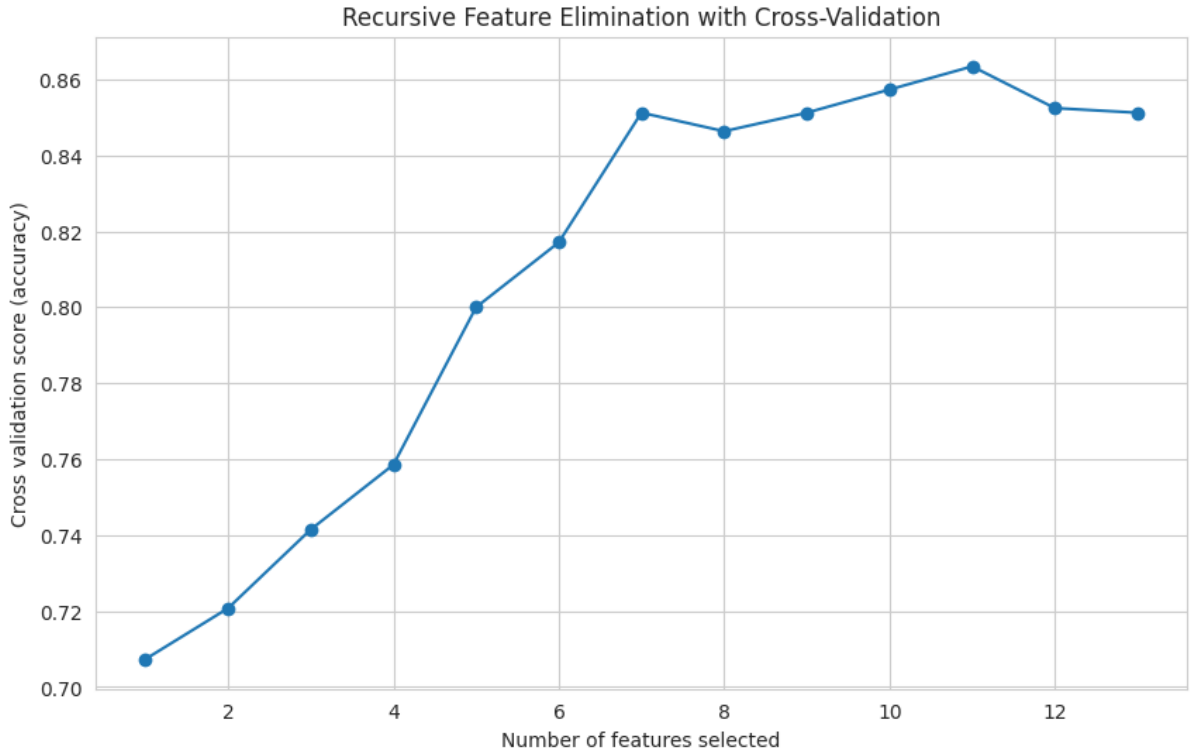



Figure 4: Recursive Feature Elimination with Cross-Validation (RFECV) accuracy score vs. number of features.

- Optimal Subset: As shown in the plot, the cross-validation score increases steadily and peaks at 11 features.

- Implications: The fact that RFECV selects nearly the entire feature set (11 out of 13) implies that the dataset has low feature redundancy; almost every feature contributes some marginal gain to the predictive power. It also suggests that for this specific sample size, ”pruning” features too aggressively hurts generalization, unlike in scenarios with massive redundancy where performance plateaus early.

## **8.4** Embedded Method: L1 Regularization (Lasso)

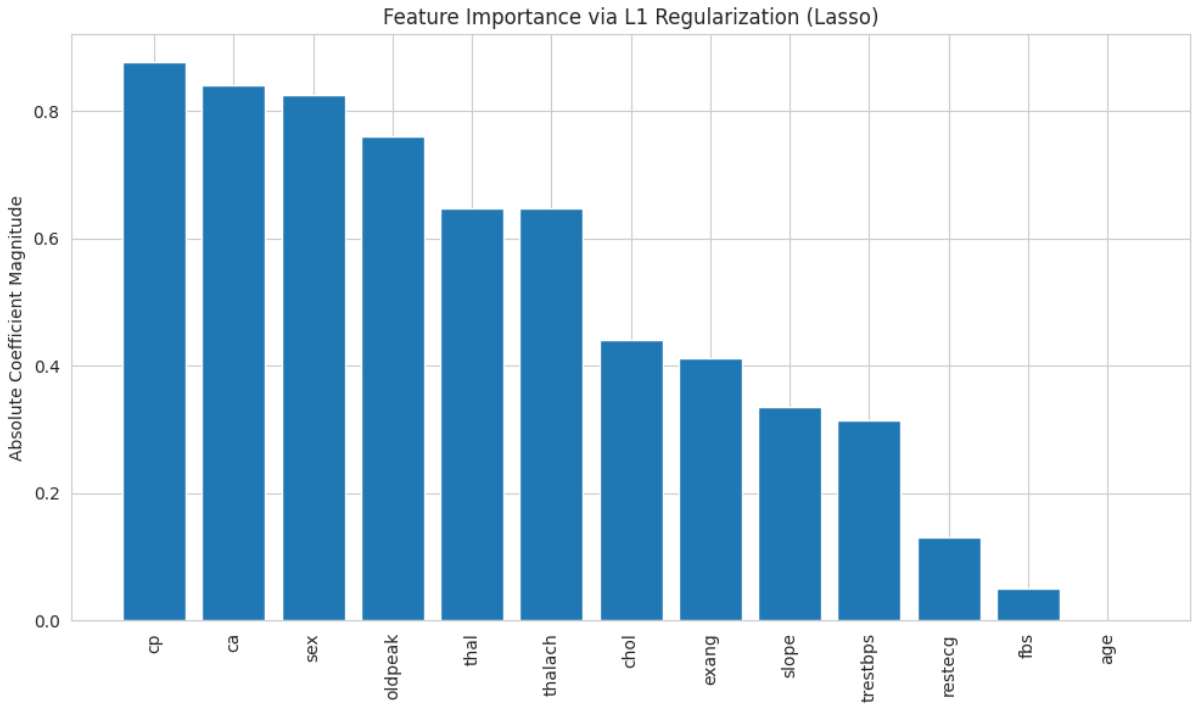



Figure 5: Feature importance magnitude derived from L1-regularized Logistic Regression (Lasso).

- Sparsity: The L1 penalty successfully drove the coefficient for age to zero, effectively removing it.
- Ranking: The importance ranking aligns well with RFE. The top features (cp, ca, sex, oldpeak) have the largest absolute coefficients. This confirms that ’Chest Pain Type’ and ’Number of Major Vessels’ (ca) are the most critical determinants for the model.

## **8.5** Dimensionality Reduction: PCA Analysis

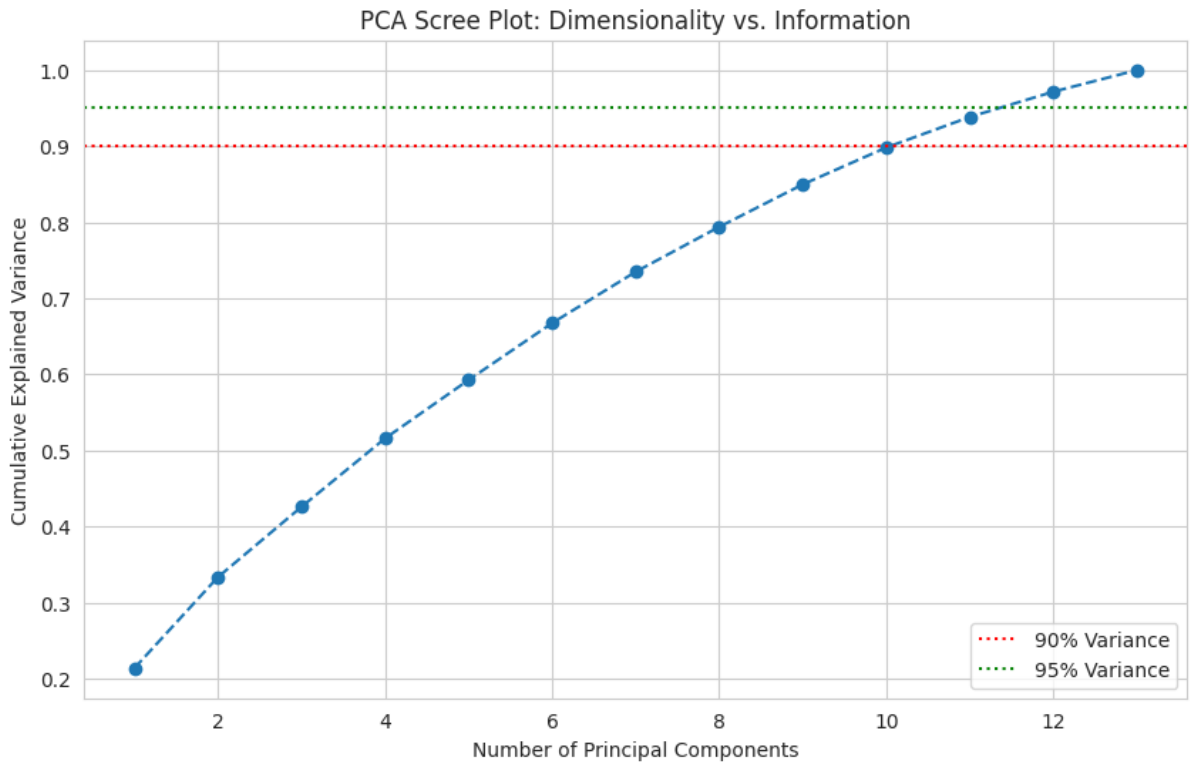



Figure 6: PCA Scree Plot showing cumulative explained variance.

- Variance Explanation: To explain 95% of the variance, PCA requires 12 components(nearly the full dimensionality of 13).
- Comparison with Feature Selection:
  - Compression: PCA fails to compress the data effectively here (linear dimensionality reduction is inefficient for this structure).
  - Interpretability: PCA components are linear combinations of all original features, making them medically uninterpretable. In contrast, RFE and Lasso preserve the original physical meanings , which is crucial for clinical decision-making.

## 9. Question 9: Image Segmentation (Bonus)

In this task, we performed image segmentation using two unsupervised clustering algorithms: K-Means and Gaussian Mixture Models (GMM). The feature vector for each pixel consisted of 5 dimensions: normalized spatial coordinates $(r, c)$ and normalized color values $(R, G, B)$.

### 9.1 K-Means Segmentation Results

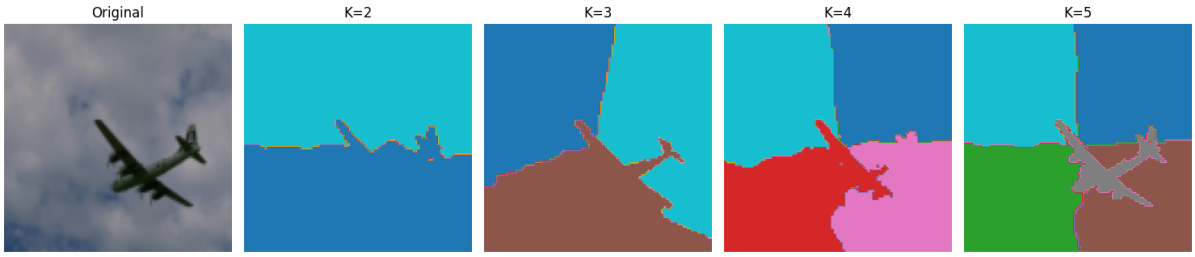



Figure 7: K-Means segmentation results for $K \in \{2, 3, 4, 5\}$. The algorithm creates distinct, hard boundaries between regions based on Euclidean distance in the 5D feature space.

## **9.2** GMM Segmentation Results

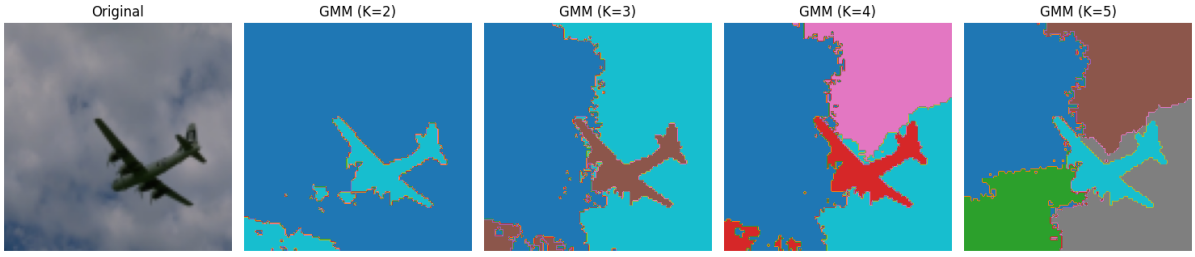



Figure 8: Gaussian Mixture Model (GMM) segmentation results for $K \in \{2, 3, 4, 5\}$. GMM assigns pixels based on maximizing the posterior probability, allowing for more flexible cluster shapes.

## **9.3** Discussion and Comparison

Comparing the visual results between K-Means and GMM:

- Boundary Characteristics (Hard vs. Soft):
  - K-Means produces very sharp, almost linear boundaries (Voronoi-like tessellations), especially visible in the sky region for $K = 3$ and $K = 5$. This is because K-Means assumes clusters are spherical with equal variance and relies strictly on Euclidean distance.

  - GMM produces more organic, irregular boundaries. For example, in the $K = 3$ GMM result, the segmentation around the airplane's edges and the clouds is more detailed. This is because GMM accounts for the *covariance* of the data, allowing it to model elliptical clusters and adapt to the varying spread of color/spatial features.

- Sensitivity to Spatial Features:
  - Both methods successfully group the sky and the airplane separately at $K = 2$.
  - However, at higher $K$ values (e.g., $K = 4$), K-Means tends to over-segment the large uniform sky area into geometric blocks simply to minimize variance. GMM handles this better by modeling the sky as a single wide distribution, though it still splits it due to the color gradient in the clouds.

- Noise and Texture:
  - The GMM results appear slightly "noisier" or more speckled at the boundaries (visible in $K = 4$ pink/blue regions). This reflects the probabilistic nature of the assignment, where pixels with ambiguous color values (e.g., at the edge of a cloud) might have competing probabilities for multiple clusters.